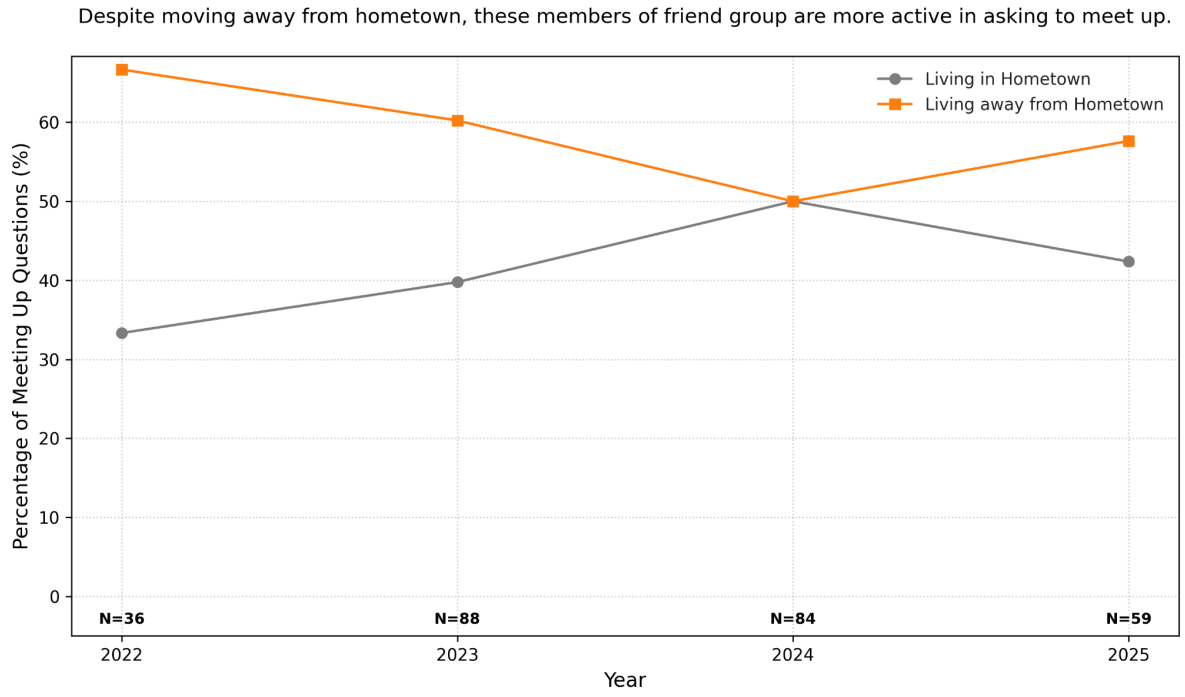
📈 Data Analysis and Visualization (DAV) Project

This project is an end-to-end Python solution for processing, cleaning, feature engineering, and analyzing whatsapp text-data, culminating in a series of data visualizations. The author (Bas) has created a set of visualisations for the course 'Data Analysis & Visualisation'. These visualisations are ready for review in the img/final-folder. The interpretation of these graphs will be explained in the next section. After that you can find a clear overview of the project structure, a clear step-by-step on how to use this project, and an overview of the pipeline flow. Please note that the visualisations presented by the author in this project are based on the group chat with his friends. The visualisations in this project have the goal to show interesting trends in the chat data based on the various features to either the messages or the attributes to the authors in the group.

Author: Bas hooge Venterink Student number: 1905776

Interpretation of the Final Images

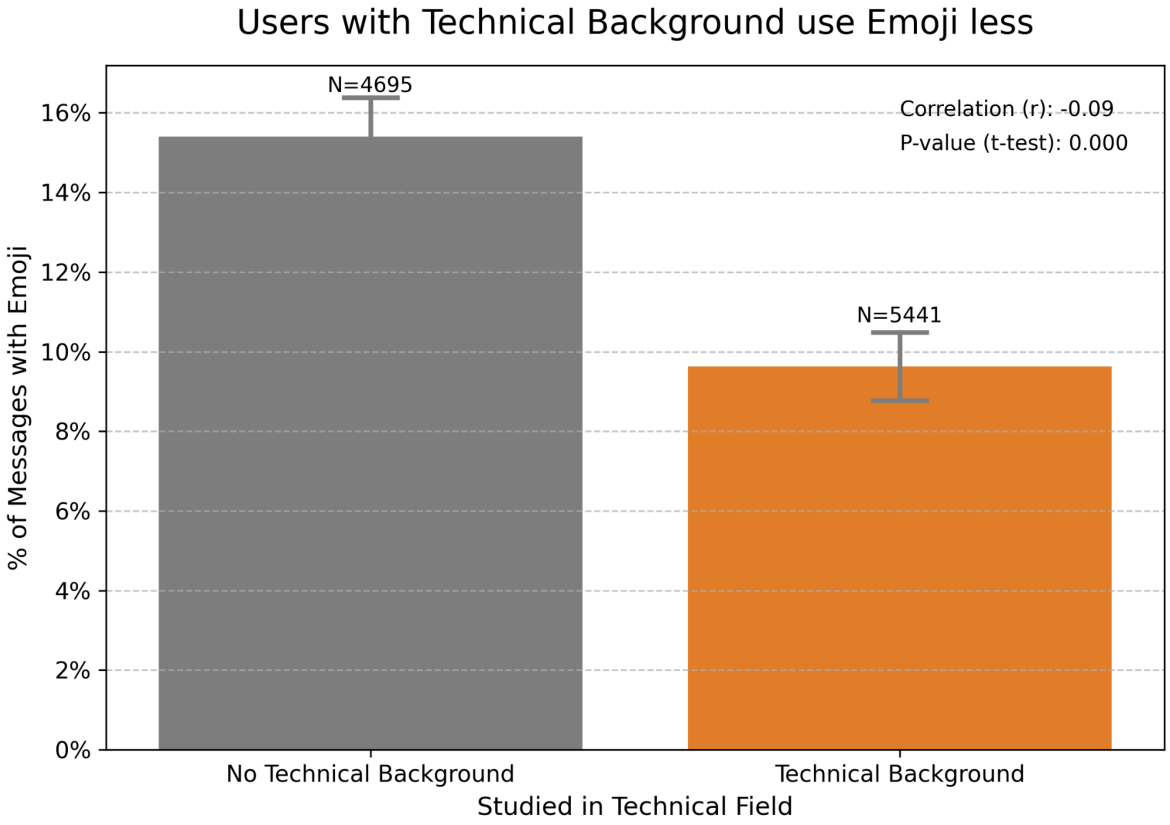
1. Categories Plot



This plot shows what percentage of each of the two groups (members of the friend group who live in the hometown where they have met and who do not live there anymore) ask the questions to meet up per year. A hypothesis could be that as friends move away from their hometown, taking initiative in meeting up will shift more to those who are still living in the hometown. This graph

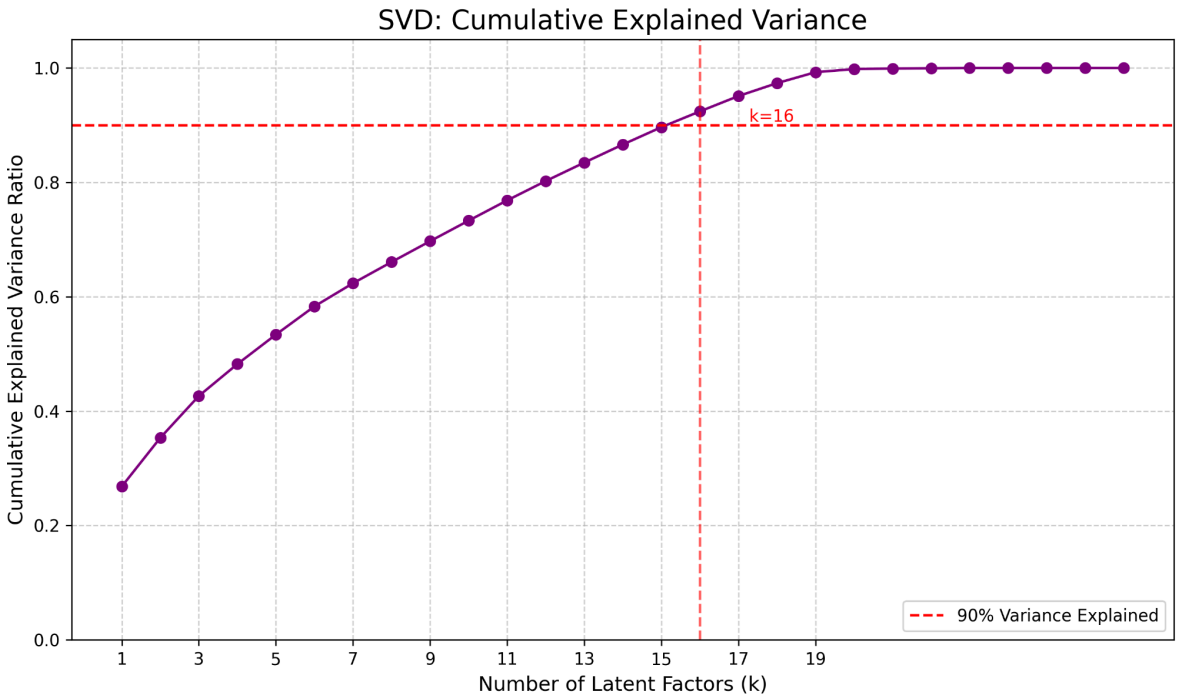
shows the opposite results for this friend group, as the majority of the questions with the intention of meeting up come from those who have moved away from their hometown.

2. Correlation Plot



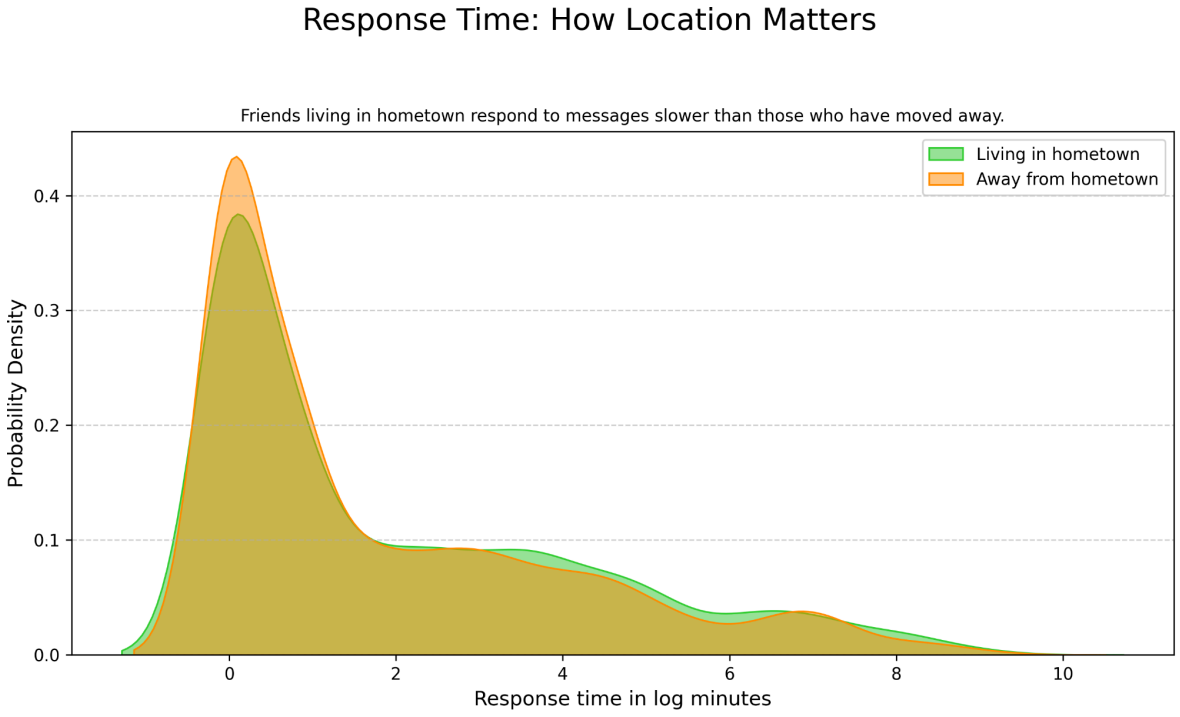
Within the friend group of the author, there is a nice even distribution of members who have performed a form of a technical study (4 out of 9 members). Based on this feature of the author of the messages, the correlation_plot.png shows that members with a technical background use an Emoji less often in their text messages (±9.7%) compared to their counterpart (±15.7%). Although the low p-value support the confidence in the title of the graph, the magnitude of the correlation between technical background and the use of emoji is very small (-0.09).

3. Dimensionality Plot



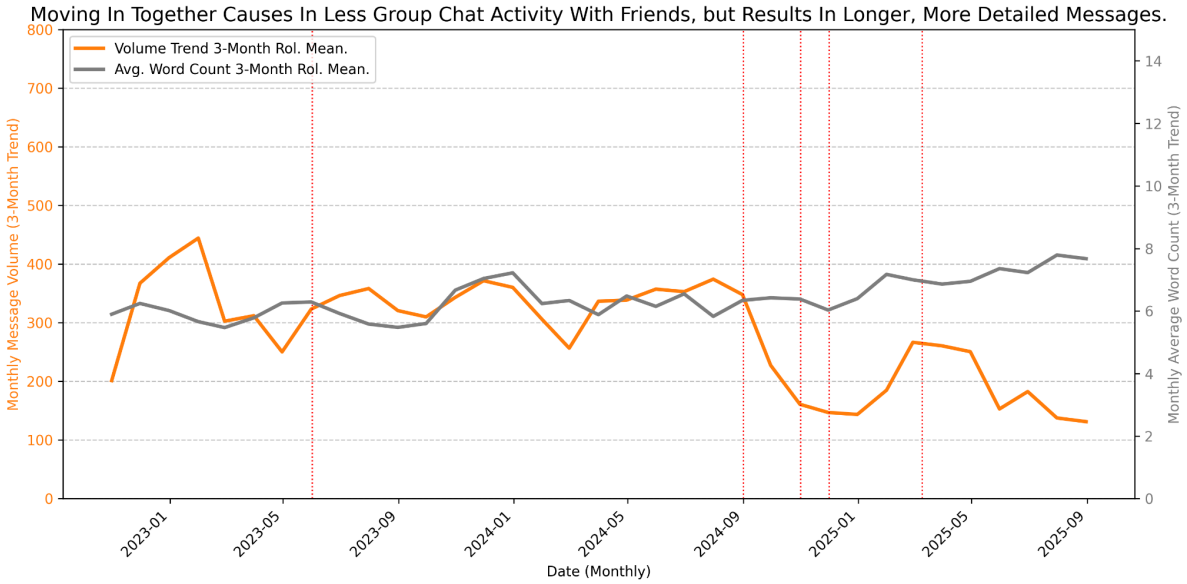
This assignment has been a bit difficult to understand and the author did not succeed in presenting an easy to interpet dimensionality reduction of the data. This graph however shows that, if the objective is to reduce the dimensionality of the data while retaining 90% of its original information, the recommended number of latent factors to choose is 16.

4. Distribution Plot



This graph aims to display response times between two groups within the friend group of the author. The group is divided in authors who have and have not been living in the hometown for the entirety of the time-period of this dataset. An hypothesis might be that people who live in the hometown might have the tendency to respond quicker to messages within the group chat, however this graph shows the opposite story. When looking at the probability density of the log response times in minutes and make a plot for each of the two groups, then we see the probability density of the group living away from hometown is higher skewed towards to 0 log minutes compared to its counterpart.

5. Time Series Plot



This graph displays the trend in the number of messages and the average size of the messages per year-month. Besides there are events displayed with the dashed, vertical red lines which showcases that in the number of members moving in their partner has increased drastically since the end of 2024. Alongside with these events, the number of texts per month drastically decreases as more members of the group are moving in with their partner. This indicates that, as more people are moving in with their partner, the social activity within the group chat decreases.

📁 Project File Structure

The project is organized to clearly separate data, source code (modules), and output images.

Folder/File	Purpose
`data/`	Contains all data files.
`├── raw/`	The initial, untouched dataset files.
`├── preprocessed/`	Datasets after preprocessing the raw txt-file to csv-file
`├── cleaned/`	Datasets after validation and cleaning routines.
`└── feature_added/`	The final dataset ready for analysis, including new features.
`src/`	The main source code directory.
`├── data_handling/`	Modules for all data preparation steps.
`│ ├── preprocess.py`	Initial structuring of raw data.
`│ ├── clean_data.py`	Data validation and cleaning.
`│ └── add_features.py`	Creation of new, derived features.
`└── graphs/`	Modules for data analysis and plot generation.
`├── time_series.py`	Time series analysis and plotting.
`├── categories_graph.py`	Categorical data analysis and plotting.

├── `distribution_graph.py`	Distribution analysis and plotting.
├── `correlation_graph.py`	Correlation analysis and plotting.
└── `dimensionality_graph.py`	Dimensionality reduction (SVD/PCA) and plotting.
`img/`	Stores the final generated visualizations.
└── `final/`	All final analysis plots (PNGs).
`config.toml`	Configuration file defining file paths and settings for the pipeline.
`main.py`	The primary execution script. Orchestrates the entire workflow.

🚀 Getting Started

This guide will walk you through cloning the repository and setting up the environment using uv, a modern, high-performance Python package manager.

1. Clone the Repository

1. Go to the project's GitHub page: `https://github.com/bashoogeventerink-max/DAV`
2. Click on the `< > Code` button and copy the HTTPS link.
3. Open VS Code.
4. Click on the "Clone Git Repository" option in the Explorer view or Command Palette (`Ctrl+Shift+P` or `Cmd+Shift+P`).
5. Paste the HTTPS link and choose a local directory to clone the project into.
6. Choose to open the file and click 'Yes, I trust the authors'.

2. Environment Setup

The project uses `uv` for dependency management.

1. Open a New Terminal in VS Code.
2. Check if `uv` is installed by typing:

```
which uv
```

3. If the output is `'uv not found'` or similar, install it using the following command (for Linux/macOS):

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

4. *If you are on Windows, please refer to the official uv installation guide.*
5. Create the virtual environment and install all project dependencies (including development dependencies) by running:

```
uv sync --all-extras
```

6. This command will create a virtual environment (`.venv` folder) and install all required packages.
7. Select UV Virtual Environment in VS Code: Open the Command Palette in VS Code (press Ctrl+Shift+P or Cmd+Shift+P).
   Type and select "Python: Select Interpreter".
   Choose the interpreter path that points to the .venv folder in your project. It will usually look something like:

```
Python X.X.X (.venv)
```

8. Where X.X.X is the Python version.
   Once selected, the environment name (.venv) should appear in the bottom-left corner of your VS Code window, and your imports should start resolving if the packages are present.

3. Run the Project

Once the environment is set up, you can execute the main script by clicking on src -> dav_bas_hv -> main.py and click on the 'Run Python-file'-button in the upright corner. This wil execute the path

Once main.py is finished, you can view the files in data -> cleaned, feature_added, preprocessed etc to see the result of the script. The graphs for the txt-file of the author is already stored in the destination directory (img -> final), so the main-script will not execute. Once you remove the images from this folder or adjust the name in config.toml for these png's, the main.py will also execute the functions to generate the graphs.

📦 Project Workflow

The project's execution is managed by `main.py`, which follows a sequential, checkpointed workflow. This design ensures that steps are only executed if their output files are not found, making re-runs efficient.

Pipeline Steps

The workflow proceeds in the following order:

1. Configuration Loading
    - Action: Loads file paths and settings from `config.toml`.
    - Output: Defines all input/output file names and target paths.
2. Preprocessing (via `src/data_handling/preprocess.py`)
    - Input: Raw data from `data/raw/`.
    - Check: Checks for `data/preprocessed/<preprocessed_csv>`.
    - Output: Generates a structured CSV file in `data/preprocessed/`.
3. Cleaning (via `src/data_handling/clean_data.py`)
    - Input: Preprocessed data.
    - Check: Checks for `data/cleaned/<cleaned_csv>`.
    - Output: Generates a cleaned CSV file in `data/cleaned/`.
4. Feature Engineering (via `src/data_handling/add_features.py`)
    - Input: Cleaned data.
    - Check: Checks for `data/feature_added/<feature_engineered_csv>`.
    - Output: Generates the final, analysis-ready CSV with new features in `data/feature_added/`.
5. Analysis & Plot Generation
    - Input: Feature-engineered data.
    - Check: Checks for respective PNG files in `img/final/`.
    - Actions: Runs five distinct analysis scripts, each generating a visualization:
        - Time Series Analysis (`time_series.py`)
        - Categories Analysis (`categories_graph.py`)
        - Distribution Analysis (`distribution_graph.py`)
        - Correlation Analysis (`correlation_graph.py`)
        - Dimensionality Analysis (`dimensionality_graph.py`)
    - Output: All final plots are saved to the `img/final/` folder.

Checkpointing Mechanism

The `main.py` script employs a simple file existence check (`.exists()`) before running any long-running task.

Example: If the Time Series plot (`img/final/time_series_plot.png`) already exists, the script will print a message and skip the `time_series_main` function, moving directly to the next analysis step. This ensures that only necessary tasks are re-executed.